

1. What is Relational Model? It was proposed by E.F. Codd in 1970 to model data in the form of relations or tables since then it was the most widely used data model and in fact, the only used database management system today in the world.

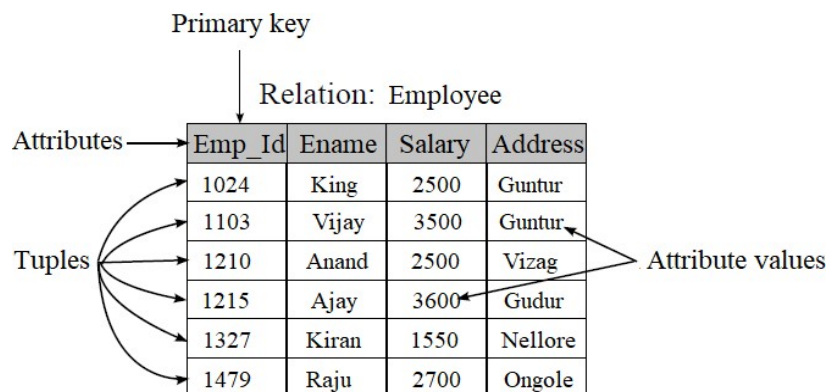
It represents how data is stored in Relational Databases. **A relational model stores data in the form of relations (tables).** Each relation has **columns** and **rows** which are formally called **attributes** and **tuples** respectively. Each **tuple** in relation is a **real-world entity**.

Relational Model Diagram

The basic structure of a relational model is tables. So, the tables are also called relations in the relational model.

The figure below will help you identify the relation, attributes, and tuples in a relational model.

It is an **Employee** relation and it is having entries of 6 employees (tuples) in it.



Terminology

- **Relations / Tables** – Relations are saved in the table format. A table has two properties rows and columns. Rows represent records and columns represent attributes.
- **Attribute:** A column header is called an attribute. A column of a relation designated by name. The name associated should be meaningful. Each attributes associates with a domain.
- **Tuple:** A single row of a table, which contains a single record for that relation, is called a tuple.
- **Degree:** The total number of attributes in a relation.
- **Cardinality:** Total number of rows present in the Table.
- **Domain:** The data type describing the types of values that can appear in each column is called a domain. Set of possible values for an attribute.
- **Relation instance:** A finite set of tuples in the relational database system represents relation instance. Relation instances do not have duplicate tuples.
- **Relation Schema:** This describes the relation name (table name), attributes and their names. A relation schema denoted by R is a list of attributes (A1, A2, ..., An) The degree of the relation is the number of attributes of its relation schema.
- **Relation key:** Each row has one or more attributes which can identify the row in the relation (table) uniquely, is called the relation key.

Characteristics

- Each relation in a database must have a distinct or unique name.
- Each attribute must have a distinct name (no two attributes should have the same name).
- Duplicate tuples must not be present in a relation.
- Each tuple must have exactly one data value for an attribute.
- No ordering is required for tuples, similarly for attributes also.

Advantages

- **Simplicity:** This model is simpler as compared to the network and hierarchical model.
- **Scalable:** This model can be easily scaled as we can add as many rows and columns we want.

- **Structural Independence:** We can make changes in database structure without changing the way to access the data. When we can make changes to the database structure without affecting the capability to DBMS to access the data we can say that structural independence has been achieved.
- **Easy to use:** This model in DBMS is easy as tables consisting of rows and columns are quite natural and simple to understand.
- **Query capability:** It makes possible for a high-level query language like SQL to avoid complex database navigation.
- **Data independence:** The Structure of Relational database can be changed without having to change any application.

Disadvantages

- Few relational databases have limits on field lengths which can't be exceeded.
- Relational databases can sometimes become complex as the amount of data grows, and the relations between pieces of data become more complicated.
- Complex relational database systems may lead to isolated databases where the information cannot be shared from one system to another.

2. Importance of Null values: IN sql, NULL is used to represent a missing value, but that it usually has one of three different interpretations—value unknown (exists but is not known), value not available (exists but is purposely withheld), or value not applicable (the attribute is undefined for this tuple).

■ Consider the following examples to illustrate each of the meanings of NULL.

- Unknown value. A person's date of birth is not known, so it is represented by NULL in the database.
- Unavailable or withheld value. A person has a home phone but does not want it to be listed, so it is withheld and represented as NULL in the database.
- Not applicable attribute. An attribute LastCollegeDegree would be NULL for a person who has no college degrees because it does not apply to that person.
- It is often not possible to determine which of the meanings is intended; for example, a NULL for the home phone of a person can have any of the three meanings.
- Hence, SQL does not distinguish between the different meanings of NULL.
- When a NULL is involved in a comparison operation, the result is considered to be UNKNOWN (it may be TRUE or it may be FALSE).
- Hence, SQL uses a three-valued logic with values TRUE, FALSE, and
- UNKNOWN instead of the standard two-valued (Boolean) logic with values TRUE or FALSE.
- It is therefore necessary to define the results (or truth values) of three-valued logical expressions when the logical connectives AND, OR, and NOT are used.
- We usually use IS NULL and IS NOT NULL operators to compare with NULL values.

Table 5.1 Logical Connectives in Three-Valued Logic

(a)	AND	TRUE	FALSE	UNKNOWN
	TRUE	TRUE	FALSE	UNKNOWN
	FALSE	FALSE	FALSE	FALSE
	UNKNOWN	UNKNOWN	FALSE	UNKNOWN
(b)	OR	TRUE	FALSE	UNKNOWN
	TRUE	TRUE	TRUE	TRUE
	FALSE	TRUE	FALSE	UNKNOWN
	UNKNOWN	TRUE	UNKNOWN	UNKNOWN
(c)	NOT	TRUE	FALSE	UNKNOWN
	TRUE	FALSE	TRUE	UNKNOWN
	FALSE	TRUE	FALSE	UNKNOWN
	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

Syntax:

- select col_names from tablename where col_name IS NULL;
- select col_names from tablename where col_name IS NOT NULL;

3. Integrity Constraints: They are referred to as conditions that are applied to the relation/table to ensure the correctness of the data in the database. These constraints are checked before performing any operation in database. If there is a violation in any of constraints, operation will fail. Thus, integrity constraint is used to guard against accidental damage to the database.

There are mostly divided into three main categories are:

- a) Domain Constraints
- b) Entity Integrity Constraints
- c) Referential Integrity Constraints
- d) Key Constraints

a. Domain Constraint

- Domain constraint defines the **domain** or **set of possible values for an attribute**.
- It specifies that the value taken by the attribute must be from its domain only.
- **Example:** Consider the following **Student** table
- Here, value '**Z**' is not allowed since only positive integer values can be taken by the **age** attribute.

Roll_no	Name	Age
S001	Akshay	20
S013	Krishna	23
S056	Vivek	Z
S054	Raghu	21

b. Entity Integrity constraint: Entity integrity constraint ensures that the primary key attribute in a relation, should not accept a null value. This is because the primary key attribute value uniquely defines an entity in a relation.

Example: Consider the following Student table

Roll_no	Name	Age
S001	Akshay	20
S013	Krishna	23
S056	Vivek	20
	Raghu	21



Not allowed, as primary key can't contain a NULL value.

c. Referential Integrity Constraints: A referential integrity constraint is specified between two tables to establish a relationship/link between them. They are implemented in the name of foreign keys.

It states that if a foreign key exists in a relation, then either **the foreign key value must match a primary key value of some tuple in its parent relation** or **its value must be null**.

The rules are:

- You can't delete a record from a primary table if matching records exist in a child table.
- You can't change a primary key value in the primary table if that record has child records.
- You can't enter a value in the foreign key field of the child table that doesn't exist in the

primary key of the parent table.

- However, you can enter a Null value in the foreign key, specifying that there are records are unrelated.

Example: Consider 2 relations “Dept” and “Emp” where “D_No” is the primary key in the “Dept” relation and foreign key in the “Emp” relation.

In this, if a foreign key in **Emp table** refers to the Primary Key of **Dept table**, then every **value of the Foreign Key** in Emp table must be **null** or **be available in Dept Table**.

(Emp Table)

EMP_NAME	NAME	AGE	D_No
1	Jack	20	11
2	Harry	40	24
3	John	27	18
4	Devil	38	13

Foreign key

Not allowed, as D_No 18 is defined as Primary key in Dept Table and in Emp Table D_No is defined as a foreign key

Relationships

(Dept Table)

Primary Key

<u>D_No</u>	D_Location
11	Mumbai
24	Delhi
13	Noida

d. Key Constraints: A Key Constraint is a statement that a certain minimal subset of the fields of a relation is a unique identifier for a tuple.

There are 4 types of key constraints-

- Super Key
- Candidate key.
- Primary key
- Foreign key.

Example: Let us understand the key constraints with the two relations **STUDENT** and **STUDENT-COURSE**.

Relation: **STUDENT**

STUD_NO	NAME	AGE	Mobile_No
512	Akshara	20	9876723452
513	Krishna	23	9991165674
546	Vivek	20	7898756543
557	Raghu	21	8987867898
595	Krishna	22	9876543210

Relation: **STUDENT-COURSE**

STUD_NO	COURSE_NO	COURSE_NAME
512	C1	DBMS
546	C2	PYTHON
512	C2	PYTHON

- **Candidate Key**

- The **minimal set of attributes which can uniquely identify a tuple is known as a candidate key**. For example, STUD_NO in STUDENT relation.
- A candidate key can never be **NULL** or **empty** and its value should be unique.
- There can be more than one candidate keys for table. For example, **STUD_NO**, as well as **MOBILE_NO** both, are candidate keys for relation STUDENT.
- The candidate key can be **simple** or **composite**. For Example, {**STUD_NO, COURSE_NO**} is a composite candidate key for relation STUDENT_COURSE.

- **Super Key**

- The set of attributes which can uniquely identify a tuple is known as Super Key. For Example, **STUD_NO, (STUD_NO, STUD_NAME), MOBILE_NO**, etc.
- Adding **zero or more** attributes to candidate key generates a super key.
- Super Key is a **super set** of Candidate key.
- A candidate key is a super key but vice versa is not true.

- **Primary Key**

- There can be more than one candidate key in a relation out of which one can be chosen as a primary key.

For Example, **STUD_NO** as well as **MOBILE_NO** both are candidate keys for relation STUDENT but **STUD_NO** can be chosen as primary key.

- **Secondary or Alternative key**

- The candidate key other than the primary key is called an alternate key / Secondary key.

For Example, **STUD_NO**, as well as **MOBILE_NO** both, are candidate keys for relation STUDENT but **STUD_PHONE** will be **alternate key**.

- **Foreign Key**

If an attribute can only take the values which are present as values of some other attribute, it will be a foreign key to the attribute to which it refers.

The relation which is being referenced is called **referenced relation** and the corresponding attribute is called **referenced attribute** and the relation which refers to the referenced relation is called **referencing relation** and the corresponding attribute is called **referencing attribute**.

The referenced attribute of the referenced relation should be the primary key for it. For Example, **STUD_NO** in **STUDENT_COURSE** is a **foreign key** to **STUD_NO** in **STUDENT** relation.

Foreign key can be **NULL** as well as may contain **duplicate tuples** i.e., it need not follow uniqueness constraint.

For Example, **STUD_NO** in **STUDENT_COURSE** relation is not unique. It has been repeated for the first and third tuple. However, the **STUD_NO** in **STUDENT** relation is a primary key and it needs to be always unique and it cannot be null.

4. Operations in Relational Model

Four basic operations performed on relational database model are

- **Insert** is used to insert data into the relation
- **Delete** is used to delete tuples from the relation.
- **Modify** allows you to change the values of some attributes in existing tuples.
- **Select** allows you to retrieve a specific range of data.

Whenever one of these operations is applied, integrity constraints specified on the relational database schema must never be violated.

Relational Algebra

5. What is query language?

Query language which is used to store and retrieve data from the database.

(Or)

It is a specialized language for asking questions, or queries, that involve the data in a database.

There are two types of query language:

- a) Procedural Query language
- b) Non-procedural Query language

Procedural Query language:

- It describes step-by-step procedure for computing the desired answer.
- Queries are specified in an operational manner. Useful for representing execution plans.

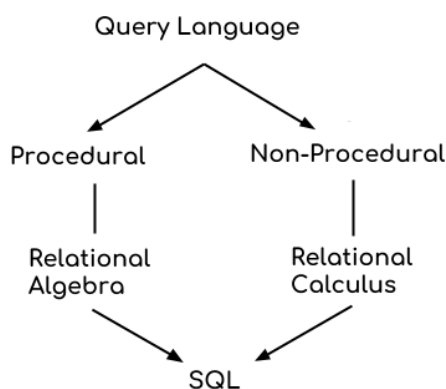
Example: **Relational Algebra** - conceptual procedural query language used on relational model.

Non - procedural Query language:

- It describes the desired answer without specifying how the answer is to be computed.
- Non - operational, declarative.

Example: **Relational Calculus** - conceptual non-procedural query language used on relational model.

For example – Let's take a real-world example to understand the procedural language, you are asking your younger brother to make a cup of tea, if you are just telling him to make a tea and not telling the process then it is a non-procedural language, however if you are telling the step-by-step process like switch on the stove, boil the water, add milk etc. then it is a procedural language.



- **Relational algebra** and **calculus** are the theoretical concepts used on relational model.
- **RDBMS** is a practical implementation of relational model.
- **SQL** is a practical implementation of relational algebra and calculus.

6. What is Relational Algebra: Relational algebra is a procedural query language that works on relational model. The purpose of a query language is to retrieve data from database or perform various operations such as insert, update, and delete on the data. When I say that relational algebra is a procedural query language, it means that it tells what data to be retrieved and how to be retrieved.

Types of operations: Two types of operations in relational algebra, they are:

— **Basic Operations**

- a) Select (σ)
- b) Project (Π)
- c) Union ($\cup$)
- d) Set Difference ($-$)
- e) Cartesian product ($\times$)
- f) Rename (ρ)

— Derived Operations

1. Natural Join ($\bowtie$)
2. Left, Right, Full outer join ($\Join$, $\Join$, $\Join$)
3. Set Intersection ($\cap$)
4. Division ($\div$)

Basic Operations

a) Select Operator (σ): It is denoted by **sigma (σ)**, and it is used to find the tuples (or rows) in a relation (or table) which satisfy the given condition.

Syntax of Select Operator (σ): **σ Condition/Predicate (Relation / Table_name)**

Where

- σ indicates selection predicate
- **Predicate/Condition** is a propositional logic formula which may use relational operators like and, or, and not.

Query: σ account_type = 'saving' (Account)

Output – It selects tuples from relation Account where the account type is 'saving'.

b) Projection (Π): It is denoted by π (Π) symbol, and it is used to select desired columns (or attributes) from a table (or relation). It is like the Select statement in SQL and syntax is:

Π column_name1, column_name2... column_nameN (Relation/ Table_name)

Where Column_name1, Column_name2, Column_nameN are attributes of Relation.

Example: We have a relation **STUDENT** with four columns; we want to fetch only two columns of the table, which we can do with the help of projection operator Π .

Query: Π std_name, std_city (Student)

Output: It selects attributes **std_name** and **std_city** from relation Student.

c) Union ($\cup$): It is used to select all the rows (tuples) from two tables (relations). It is denoted by symbol $\cup$. Its syntax is: **$R1 \cup R2$ Where $R1$ and $R2$ are the relations.**

For example, in $r1 \cup r2$, the union of two relations $r1$ and $r2$ produces an output relation that contains all the tuples of $r1$, or $r2$, or both $r1$ and $r2$, duplicate tuples being eliminated.

For a union operation to be valid, the following conditions must hold on to both relations

- **Both Tables** must be union compatible i.e., same set of attributes + domains.
- Duplicate tuples should be automatically removed.

Example 1: Consider the two relations R and S, each with two attributes A and B.

Relation: R

A	B
α	1
α	2
β	1

Relation : S

A	B
α	2
β	3

Now $R \cup S$

A	B
α	1
α	2
β	1
β	3

$R \cup S$

Example 2: To find all customers with either an account or a loan

Π customer_name (Depositor) $\cup$ Π customer_name (Borrower)

Output: It gives the **customer_name** from both relation Depositor and Borrower by eliminating duplication.

d) Set Difference ($-$): It is denoted by $-$ symbol. Let us say, we have two relation R and S, then the set difference operation $R - S$, produces a relation consisting of the tuples that are in relation R, but not in S. Both the relations R and S must be union compatible. The syntax is:

Relation_name1 $-$ Relation_name2

Example 1: Consider the two relations R and S, each with two attributes A and B.

Relation: R		Relation: S	
A	B	A	B
α	1	α	2
α	2	β	3
β	1		

Now R - S

A	B
α	1
β	1

R - S

Example 2: To find only depositors who are not taking any loan.

$\Pi[\text{customer-name (Depositor)}] - \Pi[\text{customer-name (Borrower)}]$

Output: It gives the **customer_name** which are present in relation Depositor but not in relation Borrower.

e) Cartesian product (X): It is denoted by X symbol. It is also known as Cross product. Cartesian product of the two relations (R X S) would combine each tuple of first relation R with every tuple of second relation S and its syntax is: **Relation_name1 X Relation_name2**

Note: The result set contains attributes of both the relations.

Example 1: Consider the two relations R and S, each with one attribute A and B respectively.

- The cardinality (number of tuples) of Cross Product is $m \times n$ where m is number of tuples in the first relation and n is number of tuples in the second relation.
- The degree (number of attributes) of Cross Product is $p+q$ where p is number of attributes in the first relation and q is number of attributes in the second relation.

Relation: R		Relation: S	
A		B	
α		1	
α		2	
β		3	

Now R X S

A	B
α	1
α	2
α	3
β	1
β	2
β	3

R X S

For the below result set, Cardinality = $2 \times 3 \Rightarrow 6$ Degree = $1+1 \Rightarrow 2$

- Generally, we use Cartesian product followed by a Selection operation and comparison on the operators as shown below:

σ condition (R X S)

The above query gives meaningful results.

Example 2: σ city = 'Kolkata' (Depositor X Borrower)

Output – It selects all tuples from both relations Depositor and Borrower where city is **Kolkata**.

f) Set Intersection ($\cap$): It is denoted by $\cap$ symbol and it is used to select common rows (tuples) from two tables (relations).

Let's say we have two relations R and S both have same columns, and we want to select all those tuples (rows) that are present in both the relations, then in that case we can apply intersection operation on these two relations $R \cap S$.

Note: Only those rows that are present in both the relations will appear in the result set.

Syntax: Relation_name1 $\cap$ Relation_name2

For an intersection operation to be valid, the following conditions must hold on both relations –

- **Both Relations** must be union compatible i.e., same set of attributes + domains.

Example 1: Consider the two relations R and S, each with two attributes A and B.

Relation: R		Relation: S	
A	B	A	B
α	1	α	2
β	1	β	3
α	2		

A	B
α	2

$R \cap S$

Example 2: Display customer names who are depositor as well as borrower.

Π customer-name (Depositor) $\cap \Pi$ customer-name (Borrower) **Output:** It gives the **customer_name** which are present in both the relations.

g) Rename (ρ): The results of relational algebra operations are always the relations, but they are without any name. The rename operation allows user to rename the output relation. It is denoted by using ρ (rho) and its syntax is: **ρ x (E)**

Where **E** is the expression with different relational algebra operations and **x** is the name given to their result.

Example: Let's say we have a relation CUSTOMER, we are fetching customer names and we are renaming the resulted relation to CUST_NAMES.

ρ (CUST_NAMES, Π (Customer_Name) (CUSTOMER))

Output – The output relation from the expression, Π (Customer_Name) (CUSTOMER) rename with CUST_NAMES.

h) Division operation ($/$): Consider two relation instances R and S in which R has (exactly) two fields x and y and S has just one field y, with the same domain as in R. We define the division operation R / S as the set of all x values such that for every y value in S, there is a tuple $\langle x, y \rangle$ in R.

Example that illustrates division operation:

x	y			x
S1	P1	S	y	S1
S1	P2		P2	S2
S1	P3	S1	y	S3
S1	P4		P2	S4
S2	P1		P4	
S2	P2			
S3	P2	S2	y	S1
S4	P2		P1	S4
S4	P4		P2	
			P4	

Derived Operations Join ($\bowtie$): One of the most useful operations in relational algebra. This is the most common way to combine information from two or more relations. It defined as a **cartesian product followed by selections and projections**. A Join operation **combines two tuples** from two different relations, **if and only if a given condition is satisfied**. Thus, join is a **subset** of cartesian product.

There are mainly two types of joins which are further divided as follows:

g) Inner Join

- Natural join.
- Theta join.
- Equi join.

h) Outer join

- Left Outer Join.
- Right Outer Join.
- Full Outer Join.

Inner Joins

In an inner join, only those tuples that satisfy the matching criteria are included, while the rest are excluded.

- **Natural Join ($\bowtie$):** Natural join can only be performed if there is a common attribute (column) between the relations. The name and type of the attribute must be same, and its syntax is: **R $\bowtie$ S** Where R and S are the two relations joined using natural join.

Example: We have two relations of **S (num, square)** and **C (num, cube)**. Now, we will perform natural join on both the relations i.e., **$\bowtie$ num, square, cube (S $\bowtie$ C)**

Output: The result of natural join is the set of tuples of all combinations in S and C that are equal on their common attribute names.

Relation: S

Num	Square
2	4
3	9

Relation: C

Num	Cube
2	8
3	27
4	64

Relation: S $\bowtie$ C

Num	Square	Cube
2	4	8
3	9	27

- **Theta (θ) Join:** It is denoted by the symbol θ and it combines those tuples from different relations which satisfies the condition. It can use any conditions in the selection criteria such as $<$, $>$, $\geq$, $\leq$, etc. Its syntax is **R1 θ R2** Where R1 and R2 are relations such that they don't have any common attribute. It means $R1 \cap R2 = \Phi$.

Example: We have two relations of **Student (sid, sname, std)** and **Course (class, subject)**. Now, we will perform theta join on both the relations i.e.,

STUDENT θ student.std $\geq$ course.class COURSE

Relation: Student

sid	sname	std
1	Akshara	5
2	Mahesh	10
3	Chaitanya	7

Relation: Course

class	subject
5	Moral Science
10	Computers
8	Social

Output: The result of theta join is the set of tuples of all combinations in Student and Course that are satisfying the given condition.

sid	sname	std	class	subject
2	Mahesh	10	5	Moral Science
2	Mahesh	10	10	Computers
2	Mahesh	10	8	Social

- **EQUI join:** When a theta join uses only equivalence condition, it becomes an equi-join.

STUDENT θ student.std = course.class COURSE

Output:

sid	sname	std	class	subject
1	Akshara	5	5	Moral Science
2	Mahesh	10	10	Computers
3	Chaitanya	7	8	Social

OUTER JOIN:

In outer join, along with tuples that satisfy the matching criteria, we also include **some** or **all tuples** that do not match the criteria.

- **Left Outer Join ($\bowtie$):** This join returns all the tuples of the left relation and matching tuples for the right relation. However, if there is no matching tuple is found in right relation, then the attributes of right relation in the join result are filled with NULL values. Syntax is: **R1 $\bowtie$ R2** where R1 and R2 are relations.



Fig.: Venn diagram for Left join

Example: We have two relations of S (*num, square*) and C (*num, cube*). Now, we will perform left outer join on both the relations i.e., $S \bowtie C$

Relation: S

num	square
2	4
3	9
5	25

Relation: C

num	cube
2	8
3	27
4	64

Output:

$S \bowtie C$

num	square	num	cube
2	4	2	8
3	9	3	27
5	25	-	-

- **Right Outer Join ($\bowtie$):** This join returns all the tuples of the right relation and matching tuples for the left relation. However, if there is no matching tuple is found in left relation, then the attributes of left relation in the join result are filled with NULL values. Syntax is: **R1 $\bowtie$ R2** where R1 and R2 are relations.



Fig.: Venn diagram for Right join.

Example: By considering the previous relations, we obtain the following result when we apply the right outer join on both relations. i.e., $S \bowtie C$

Output:

$S \bowtie C$

num	square	num	cube
2	4	2	8
3	9	3	27
-	-	4	64

- **Full Outer Join ($\bowtie$):** In a full outer join, all tuples from both relations are included in the result, irrespective of the matching condition. The rows for which there is no matching, the result-set will contain NULL. Syntax is: $R1 \bowtie R2$ Where R1 and R2 are relations.



Fig.: Venn diagram for Full join.

Example: By considering the previous relations, we obtain the following result when we apply the full join on both relations. i.e., $S \bowtie C$ Output:

$S \bowtie C$

num	square	num	cube
2	4	2	8
3	9	3	27
5	25	-	-
-	-	4	64

7. **Relational Calculus:** It is a non-procedural query language that tells the system what data to be retrieved but doesn't tell how to retrieve it. It only focuses on what to do, and not on how to do it. It exists in two forms:

- Tuple Relational Calculus (TRC)
- Domain Relational Calculus (DRC)

a) **Tuple Relational Calculus (TRC):** It is used for selecting those tuples that satisfy the given condition. In Tuple Calculus, a query is expressed as: $\{t \mid P(t)\}$

Where t = resulting tuples

$P(t)$ = known as **Predicate** or is the condition used to fetch t .

Note: $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT ($\neg$).

For example: $\{T.name \mid \text{Student}(T) \text{ AND } T.address = 'Guntur'\}$

Output: This query selects the tuples from the STUDENT relation. It returns a tuple with 'name' from Student whose address is 'guntur'.

b) **Domain Relational Calculus (DRC):** In domain relational calculus, filtering is done based on the domain of the attributes and not based on the tuple values. In Domain Calculus, a query is expressed as: $\{c1, c2, c3, \dots, cn \mid P(c1, c2, c3, \dots, cn)\}$

Where $c1, c2, \dots$ etc. represents domain of attributes (columns).

P defines the formula including the condition for fetching the data.

For example: $\{<name, address> \mid \in \text{Student A address} = 'guntur'\}$

This query selects the tuples from the STUDENT relation. It returns attributes **name, address** from Student whose address is 'guntur'.

Example: Consider the Sailors-Boats-Reserves database described in the text.

Sailors (*sid*: integer, *sname*: string, *rating*: integer, *age*: real)

Boats (*bid*: integer, *bname*: string, *color*: string)

Reserves (*sid*: integer, *bid*: integer, *day*: date)

Write each of the following queries in Relational Algebra:

- Display the details of sailors who have rating greater than 8.
- Display the details of red color boats.
- Display the name and age of all sailors.
- Find names of sailors who have ratings at least 8.
- Find the names of sailors who have reserved boat 103.
- Find the *sids* of sailors who have reserved green boat.
- Find the names of sailors who have reserved a red boat.
- Find the colors of boats reserved by Lubber.
- Find the names of boats reserved by Dustin.

Relation: Sailors

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zobra	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Relation: Reserves

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Relation: Boats

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

- Display the details of sailors who have rating greater than 8.
 $\sigma_{\text{rating} > 8}(\text{SAILORS})$
- Display the details of red color boats.
 $\sigma_{\text{color} = \text{'red'}}(\text{BOATS})$
- Display the name and age of all sailors.
 $\Pi_{\text{sname}, \text{age}}(\text{SAILORS})$
- Find the names of sailors who have ratings at least 8?
 $\Pi_{\text{sname}}(\sigma_{\text{rating} \geq 8}(\text{SAILORS}))$
- Find the names of sailors who have reserved boat 103?
 $\Pi_{\text{sname}}((\sigma_{\text{bid} = 103}(\text{RESERVES}) \bowtie \text{SAILORS}))$

- Find the sid of sailors who have reserved green boat.

$\Pi \text{ sid } ((\sigma \text{ color} = \text{'green'} (\text{BOATS}) \bowtie \text{RESERVES}))$

- Find the names of sailors who have reserved a red boat.

$\Pi \text{ sname } ((\sigma \text{ color} = \text{'red'} (\text{BOATS}) \bowtie \text{RESERVES} \bowtie \text{SAILORS}))$

- Find the colors of boats reserved by Lubber.

$\Pi \text{ color } ((\sigma \text{ sname} = \text{'Lubber'} (\text{SAILORS}) \bowtie \text{RESERVES} \bowtie \text{BOATS}))$

- Find the names of boats reserved by Dustin.

$\Pi \text{ bname } ((\sigma \text{ sname} = \text{'Dustin'} (\text{SAILORS}) \bowtie \text{RESERVES} \bowtie \text{BOATS}))$